OPERATING SYSTEMS MINI PROJECT

BANKER'S ALGORITHM SIMULATION FOR DEADLOCK AVOIDANCE

Submitted by: Vinisha Uttam Gavade

Roll No.: 42

Department: Artificial Intelligence and Data Science

College: Alard University Pune

Academic Year:2026-27

Abstract

Deadlock is a situation in which a group of processes waits indefinitely for resources held by one another. The Banker's Algorithm is a deadlock-avoidance technique that checks whether granting resources can leave the system in a safe state. This project implements the safety algorithm in C. It accepts the Allocation, Maximum and Available resource data, computes the Need matrix, and searches for a safe sequence. The program reports either a safe state with the sequence or an unsafe state when no complete safe sequence is found.

1. Introduction

An operating system manages processes and allocates resources such as memory, devices and files. When several processes compete for limited resources, deadlock may occur. Deadlock avoidance checks resource requests against the system's future needs before allowing an allocation. The Banker's Algorithm, introduced by Edsger W. Dijkstra, is a classic method for checking whether the current allocation is safe.

2. Problem Statement

To develop a C program that models resource allocation for multiple processes and determines whether the system is in a safe state by applying the Banker's Algorithm.

3. Aim

To implement and demonstrate the Banker's Algorithm for deadlock avoidance using C programming.

4. Objectives

• Understand the concepts of deadlock, resource allocation and safe states.

• Accept Allocation, Maximum and Available resource matrices from the user.

• Calculate the Need matrix using Need = Maximum − Allocation.

• Apply the safety algorithm to find a safe sequence.

• Display a clear result indicating a safe or unsafe state.

5. Scope

This project is an educational simulation. It checks a fixed snapshot of processes and resources supplied by the user. It does not control the actual operating system's resources or automatically grant real resource requests.

6. Theory

6.1 Key terms

• **Available:** Number of currently available instances of each resource type.

• **Max:** Maximum number of instances of each resource type a process may need.

• **Allocation:** Number of resource instances currently assigned to a process.

• **Need:** Remaining resources a process may request; Need = Max − Allocation.

• **Safe state:** A state in which there is at least one order of process completion that lets every process finish.

• **Unsafe state:** A state for which no safe sequence can be found. Unsafe does not necessarily mean deadlock has already occurred.

6.2 Safety algorithm

Set Work equal to Available and mark every process unfinished. Find an unfinished process whose Need is less than or equal to Work for every resource type. If one is found, assume it completes, add its Allocation to Work, mark it finished and append it to the safe sequence. Repeat. If every process finishes, the system is safe. If no eligible

process can be found while unfinished processes remain, the state is unsafe.

7. Requirements

Hardware

A standard desktop or laptop capable of running a C compiler.

Software

Operating system: Windows or Linux; compiler: GCC, Code::Blocks or another standard C compiler; editor: VS Code, Code::Blocks or any text editor.

8. System Design

Inputs: number of processes, number of resource types, Allocation matrix, Maximum matrix and Available vector. Processing: validate inputs, calculate Need, run the safety algorithm. Outputs: Need matrix and safe/unsafe result with a safe sequence when one exists.

Data structures

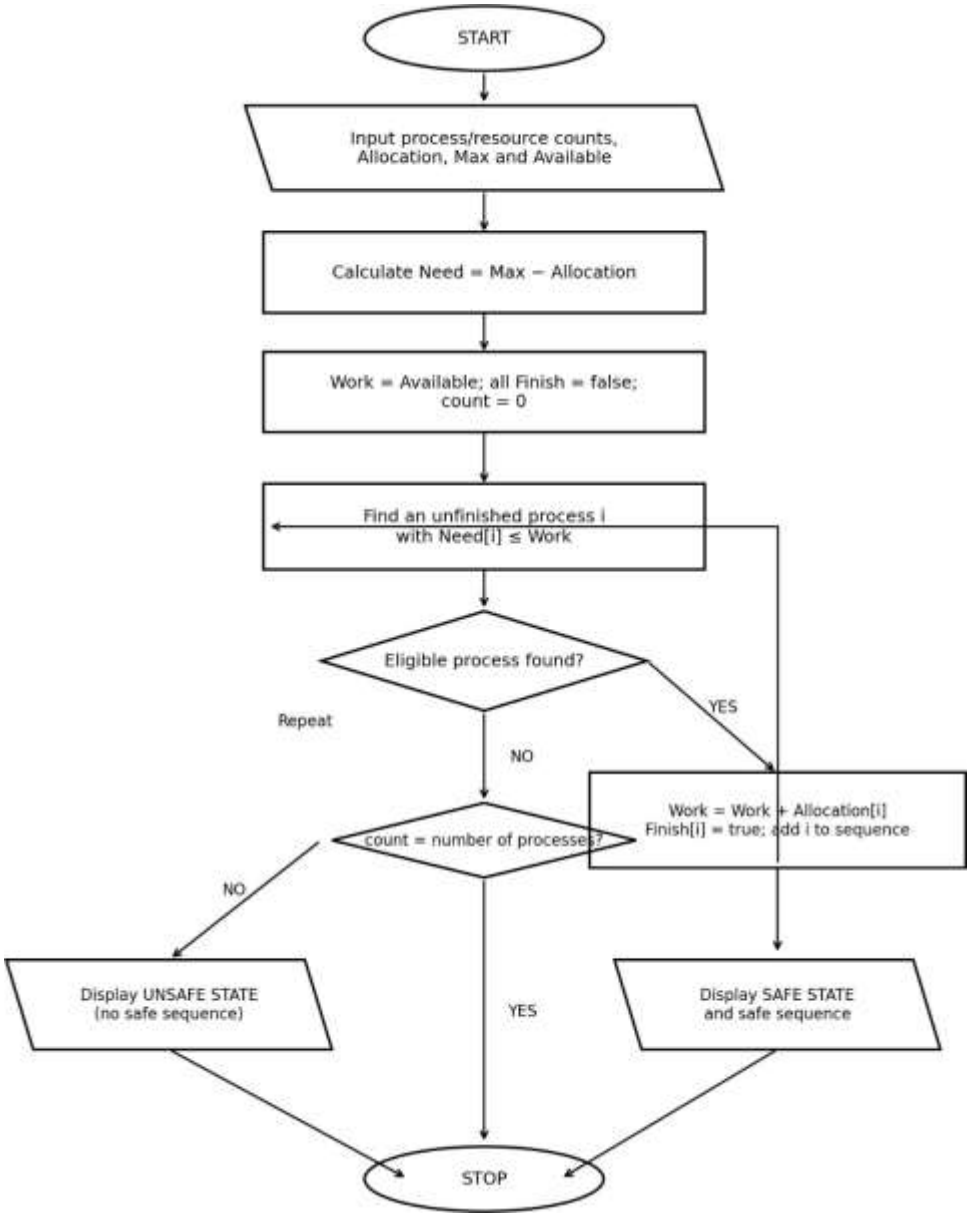
Two-dimensional integer arrays store Allocation, Maximum and Need. One-dimensional arrays store Available, Work, Finish and the safe sequence. The program supports up to 20 processes and 10 resource types.

9. Algorithm

1. Start.
2. Read the number of processes n and resource types m.
3. Read the Allocation matrix.
4. Read the Maximum matrix and verify each Max entry is at least its corresponding Allocation entry.
5. Read the Available vector.
6. Compute Need[i][j] = Max[i][j] − Allocation[i][j] for every process and resource.
7. Copy Available into Work and mark every process unfinished.
8. Search for an unfinished process i whose Need[i] is less than or equal to Work for every resource.
9. If found, set Work = Work + Allocation[i], mark the process finished and add it to the sequence; repeat the search.
10. If all processes finish, display the safe state and safe sequence. Otherwise, display the unsafe state.
11. Stop.

10. Flowchart

The flowchart below shows the input, Need calculation, safety check, resource release and final decision.



11. Source Code

The complete, compilable C source is provided separately as **bankers_algorithm.c**. The program includes input checks and comments explaining the safety algorithm. Compile with `gcc bankers_algorithm.c -o bankers` and run `./bankers` (or `bankers.exe` on Windows).

12. Sample Test Case

Number of processes: 5; number of resource types: 3.

Allocation matrix

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum matrix

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available: (3, 3, 2)

Need matrix: P0 = (7,4,3), P1 = (1,2,2), P2 = (6,0,0), P3 = (0,1,1), P4 = (4,3,1).

Expected result: Safe state. One safe sequence is P1 → P3 → P4 → P0 → P2. Other safe sequences may also be valid.

13. Testing

Test	Input condition	Expected result
1	Known sample data above	Safe state and a valid safe sequence
2	Available resources too small to finish any process	Unsafe state
3	Maximum value smaller than Allocation	Input rejected
4	Negative resource value	Input rejected
5	Invalid process/resource count	Input rejected

14. Results and Discussion

The program calculates the Need matrix and uses the safety algorithm to determine whether all processes can finish in some order. For the sample input, the system is safe and the sequence P1 → P3 → P4 → P0 → P2 is valid. If the program cannot find an eligible unfinished process before all processes complete, it reports an unsafe state. The result depends on the supplied matrices and available resource counts.

15. Advantages

• Simple way to understand deadlock avoidance.

• Shows how resource availability changes as processes finish.

• Provides a safe sequence for verification.

• Uses standard C arrays, loops and conditional statements.

16. Limitations

• The user must provide the resource data correctly.

• The simulation checks a fixed snapshot and does not handle concurrent real processes.

• The classic algorithm assumes each process declares its maximum resource need in advance.

• An unsafe state indicates that safety cannot be guaranteed; it does not prove deadlock is already happening.

17. Future Scope

The project can be extended with a graphical user interface, animated resource-allocation diagrams, automatic test-case generation, a step-by-step trace of Work and Finish values, and an option to test a new resource request before granting it.

18. Conclusion

The Banker's Algorithm Simulation demonstrates how an operating system can avoid unsafe resource allocations by checking for a safe sequence. The C implementation reinforces key Operating Systems concepts such as processes, resource allocation, matrices, deadlock avoidance and safe states. It is a useful academic mini-project for learning and demonstrating the safety algorithm.

19. References

• Silberschatz, A., Galvin, P. B., and Gagne, G., *Operating System Concepts*, Wiley.

• Stallings, W., *Operating Systems: Internals and Design Principles*, Pearson.

• Dijkstra, E. W., foundational work on deadlock avoidance and the Banker's Algorithm.

• Operating Systems course notes and laboratory material supplied by the department.

Submission note: Fill in your name, roll number, department, college and academic year on the cover page before submission.